



Hamster names

Popchop, Fuzzable and Princess POP - these are some of the names for hamsters generated by AI <https://dgit.in/sep18-49-1>



Bike Share Graveyards in China

Photos of huge piles of abandoned and broken bicycles in China, because of an oversupply problem. <https://dgit.in/sep18-49-2>

OBTW this is a comment

No it's a two line comment

Oops no.. it has many lines

TLDR

HELLO WORLD

HAI 1.2

BTW this is how initialize variables

I HAS A hui ITZ "HAI

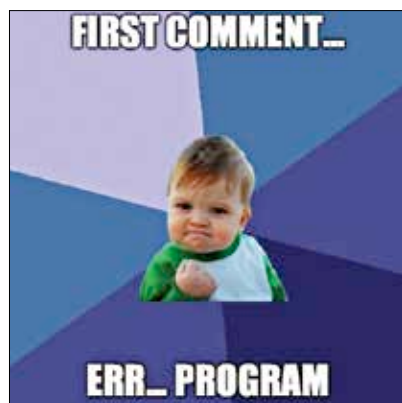
WORLDZZZ!!!1!!"

VISIBLE hui

KTHXBYE

The above program includes

1. A single line that acts as the begin control statement for the program
2. A single line comment about the program.
3. A variable `hui` that is defined and initialized to a greeting.
4. A statement to print the value of the variable `hui`.



Save the above file with a name of your choice and an extension of `!.lol`. You can then run the lolcode file by calling the command:

```
$ lci filename.lol
```

Congratulations! You have successfully written and executed your first program in lolcode, You gangsta!

TYPES

The following are the supported data types in lolcode -

- YARN -Strings
- NUMBR - Integers
- NUMBAR - floats
- TROOF - boolean
- BUKKIT - Arrays

In LOLCODE the variable type is handled on its own. If a variable type is

undeclared it is called untyped (known as NOOB).

Untyped (NOOB)

As explained before a variable whose type has not been declared is assigned this type as default, it cannot be converted into any other type except TROOF (boolean). The casting of a NOOB into TROOF makes the variable FAIL (false).

When trying to cast a NOOB data type into other types the variable results to zero values or its equivalent for all other types. To create a NOOB variable, simply declare a variable and assign a value to it:

HAI 1.2

I HAS A NOOBY

VAR3 R "SOMETHINZ"

VISIBLE NOOBY

Output of Program:

```
$ lci main.lol
```

SOMETHINZ

Booleans (TROOFS)

In most languages, boolean generally has two values- true and false. But, in LOLCODE, the boolean data type is called TROOF, and the the corresponding values for true and false are WIN & FAIL respectively. All uninitialized values like an empty string (`""`), or an empty array will all cast to FAIL.

HAI 1.2

I HAS A HONEST ITZ A TROOF

VAR3 R "FAIL"

VISIBLE HONEST

KTHXBYE

Output of the Program:

```
$lci main.lol
```

FAIL

Numerical Types (NUMBR)

In LOLCODE, the integer data type is called NUMBR. Any number of digits is considered as a NUMBR, unless it has a decimal appearing in it. Negative values of NUMBR are preceded by a hyphen (-), as shown:

HAI 1.2

I HAS A VALU ITZ A NUMBR

VISIBLE VALU

KTHXBYE

Output of the Program:

```
$lci main.lol
```

0

Float (NUMBAR)

Just like the NUMBR data type, LOLCODE has another type, which represents a decimal or a float in most programming languages. In LOLCODE, a NUMBAR represents a float containing a decimal point. Casting a NUMBAR to a NUMBR truncates the decimal portion of the floating point number and returns it as a NUMBR, without any decimal.

Strings (YARN)

All values containing strings, i.e. string literals (YARN) are classified into this data type and should start and end with double quotation marks (`""`). Anything may be written inside the string, like space, comma, full stop, exclamation or any other symbol. Escape characters are represented by colons in LOLCODE, and any value following a colon gets a special meaning.

- `:)` - A closing bracket following a colon represents a newline (`\n`)
- `:>` - A closing angle bracket following a colon represents a tab (`\t`)
- `:o` - A 'o' character following a colon represents a bell (beep) (`\g`)
- `:"` - A " following a colon represents a literal double quote (`"`)
- `::` - A colon following a colon represents a single literal colon (`:`)

HAI 1.2

I HAS A WARDZ ITZ A YARN

VAR3 R "XYZ"

VISIBLE WARDZ

KTHXBYE

Output:

```
$lci main.lol
```

XYZ

Array (BUKKIT)

This data type is the equivalent of an array. It contains named slots, which can store either variables or even functions. A BUKKIT is declared in the following way:

I HAS A [object] ITZ A BUKKIT

[object] HAS A [var] ITZ

[value]